Short Order — project brief

Nov 11, 2026 · @Viktrix

Write one Python class that decides, at every scheduling point, which cook works on which order. It is marked by simulating nine service profiles against hidden seeds, scored on six weighted components, and ranked against reference schedulers on a public leaderboard — the one to beat is `gordon_ramsay`.

The task

You write one Python file containing one class with one method, `schedule`. It is called every time something happens in the kitchen and answers a single question: which order should each cook be working on right now? Everything else — the tick, the context switch, the patience rule, the metrics, the score — lives in a compiled engine that ships with the kit, so a run on your machine obeys exactly the rules the marking server applies.

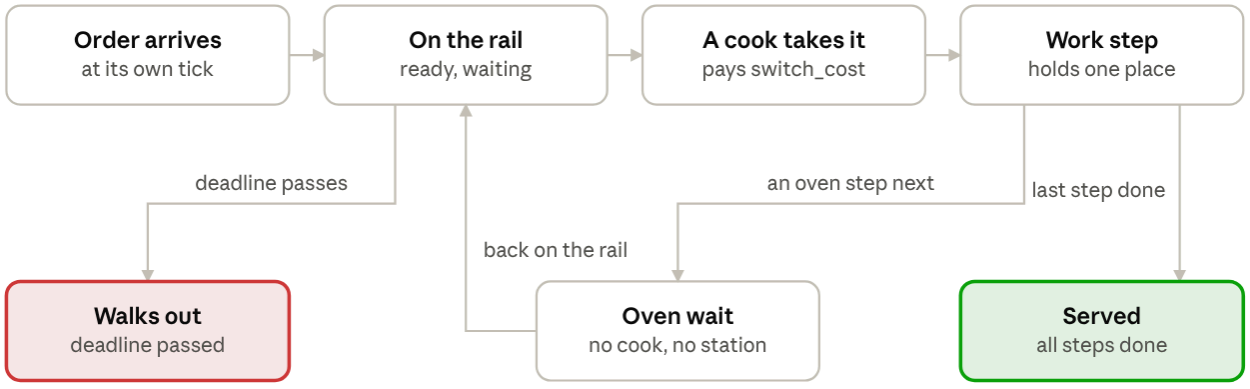
The kitchen is an operating system in costume. A cook is a CPU core, an order is a process, a recipe's `work` steps are CPU bursts and its `wait` steps (the oven) are I/O bursts, a station is a lock with a fixed number of permits, the customer's patience is a deadline, and the ticket rail is the ready queue. You are the scheduler.

The starter file already works, so the loop is: change one thing, run it, see whether the number moved.

How the kitchen works

Every order needs a cook and a place at its station; the oven needs neither

The path of one order, from the ticket rail to the pass



an order's path · 7 states, 2 endings

Time runs in ticks. Three rules give the game its shape. **Switching costs**: every time a cook takes on an order, from idle or instead of what it was doing, it spends `switch_cost` ticks before any work happens. **Customers leave**: an order not served by its deadline is abandoned, and every tick spent on it was wasted, however nearly done it was. **Stations are shared**: each station has a fixed number of places, a cook holds one for as long as it holds the order, and an assignment to an order whose station is full is refused outright, not queued — the cook simply stays idle for that decision.

A cook only ever holds one station and always lets go before taking the next, so the kitchen cannot deadlock. It can starve and convoy beautifully, and that is what you are up against. One wrinkle the drawing leaves out: when a work step ends and the next one is at a different station that is full, the order is *bumped* back to the rail rather than hold a cook hostage — and resuming it costs another switch.

What you write

Subclass `Scheduler` and implement `schedule`. Only that one method is required; `reset(seed)` and `on_run_end(result)` are optional.

```
from kitchen import Decision, Scheduler


class MyScheduler(Scheduler):
    name = "my_scheduler"
    version = "1"

    def reset(self, seed):
        """Optional. Once per run, before the first call."""

    def schedule(self, obs):
        """Required. Takes an Observation, returns a Decision."""
        return Decision()
```

You are called at **scheduling points**: when service opens, when an order arrives, when a cook finishes a step, when a dish comes out of the oven, when a customer walks out, and when an alarm you set goes off. Between those points the kitchen runs on its own and nothing you decided changes.

The `Observation` is everything you can see: `obs.ready` (the rail), `obs.cores`, `obs.stations` and `obs.free_stations()`, each order's recipe, steps, arrival and `time_left`, plus `obs.history` of what has already been served. One profile hides step durations from you entirely.

The `Decision` names cooks. A cook you do not name carries on; an order you do not name waits.

Call	What it does
`d.assign(core, order)`	Put this order on this cook
`d.idle(core)`	Take the cook off its order; the order returns to the rail
`d.wake_in(8)`	Call me again in 8 ticks even if nothing happens; `wake_in(0)` cancels
`d.annotate(...)`	Drawn by the viewer, ignored by the engine

Assigning an order to a cook that already has it changes nothing and costs no switch. Assigning it to a different cook preempts the first, which keeps its progress but means somebody pays a switch to resume it. You may move an order between cooks in one decision — the engine lets go before it dispatches.

The alarm matters more than it looks. You are only called when something happens, and a time slice running out is not something that happens on its own. Without `wake_in`, round robin is not expressible at all.

How it is scored

Six components, each scored between 0 and 1 and weighted into a score out of 100: `100 × Σ(component × weight) ÷ Σ weights`. A forfeited run scores zero whatever the components say. These are the default weights; some profiles re-weight them.

Component	Weight	What it measures	Formula
completion	30	Customers served before they left	`served / orders`
response	12.5	How quickly an order first got a cook	`1 / (1 + mean_response / 25)`
turnaround	12.5	How long a customer was in the room	`1 / (1 + mean(left − arrived) / 60)`
slowdown	15	Time taken relative to time needed	`1 / mean(turnaround / max(span, 10))`

Component	Weight	What it measures	Formula
switching	15	How much switching was *your* choice	`necessary switches / all switches`
fairness	15	How evenly slowdown was shared	Jain's index over every order's slowdown

Three of the six are latency measured three different ways, and a scheduler can win one while losing another. **Response** stops at the first tick of work, so it includes the switch — the best possible response here is `switch_cost`. **Turnaround** is the whole stay in ticks, including every oven wait and every tick back on the rail after a preemption. **Slowdown** is turnaround divided by the time the order actually needed, so a two-tick espresso that took twenty is punished far harder than a sixty-tick banquet that took eighty. Oven time sits in the denominator as well as the numerator on purpose: you are not marked down for time nobody could have saved.

Two components decide most of the design. **Completion is the big one**, and abandoned orders hurt twice — the customer is gone and the cook time spent on them was wasted, so a dish that cannot finish before its deadline is not worth starting. **Switching** counts only switches that were your choice: a resumption after you preempted something is unnecessary, while coming back from the oven or after a bump starts a fresh step and is free. A scheduler that never preempts banks that component outright.

The profiles you are marked on

A profile is a menu, a load and the rules: how many cooks, what switching costs, whether durations are shown. They live in `configs/`. Marking uses nine, and **your mark is the mean over profiles of your mean score per profile**, so no single profile can be gamed.

Profile	What it stresses
`default`	The course profile — lunch service, two cooks, a mixed menu
`rush`	Sustained overload; most of the room cannot be served at all
`one-cook`	A single cook, so any long dish monopolises the kitchen
`brigade`	Four cooks and a heavy load

Profile	What it stresses
`banquet-night`	Long, patient banquets against short, impatient orders
`bake-off`	A wide menu, heavy on oven waits
`blind`	Step durations hidden — a scheduler that needs them fails here
`service-line`	Four cooks sharing one place at the pass; ignore `obs.stations` and it shows
`function`	The whole party is seated before service opens and nothing arrives afterwards, so the kitchen stops asking you anything until a dish finishes. `wake_in` is the only way to be asked again. Weighted differently from the rest

Each profile's file opens with a comment saying what it is for, and `function`'s is worth reading closely — it is asking a different question from the others.

You are also ranked against twelve reference schedulers compiled into the engine. You can play them and measure them but not read their code; working out how they do it is the assignment. `gordon_ramsay` is the strongest and the one to beat. The classics are all there too — FIFO, round robin, SJF, SRTF, priority, EDF, stride and MLFQ — and it is worth noticing that SJF outscores SRTF here, because that gap is the whole puzzle.

Rules and limits

50 milliseconds per call. A decision that takes longer is discarded — the kitchen carries on as if you had said nothing — and counted as a timeout. The budget is generous: the reference schedulers decide in microseconds and a sensible Python scheduler in well under a millisecond. Where it bites is a search over every ordering of a fifty-order rail.

Twenty consecutive failures forfeit the run, scoring zero. A failure is a timeout, an exception, or a return value that is not a decision.

Some assignments are rejected rather than obeyed. The engine will not assign an order that is in the oven, was already served, has not arrived, or does not exist; nor one whose station is full; nor name a cook twice or put one order on two cooks. The rest of your decision still applies and the rejection is counted and shown to you. It costs nothing directly, but the cook you meant to use stays as it was — which on a profile with a tight station is most of the marks.

There is no sandbox. Your file runs as the marker's user, and the pipeline flags for a human to read any import of `subprocess`, `socket`, the network libraries, `pickle` and the like. A scheduler has no business making them.

What you hand in

One file: `my-scheduler/scheduler.py`, uploaded to Moodle. That is the whole submission. Moodle knows who you are; there is nothing to fill in.

All of your code goes in that one file. A second `.py` file beside it is **not importable** when the marker loads your scheduler — the file is loaded by path, not as a package — so it is rejected rather than silently ignored. The same goes for everything else: no `data/` folder, no notes, no README. Any table your scheduler needs must be written out inside `scheduler.py`. The file must stay under 256 KB.

Before uploading, run the marker's own first step:

```
./start.sh check my-scheduler        # macOS and Linux
START.cmd check my-scheduler         # Windows
```

It checks the folder's structure, then runs your scheduler through every public seed under the marker's failure policy. A submission that passes here is one the pipeline accepts.

Marking then runs your file on every profile over a **hidden seed set**, with the same engine, rules and failure policy you have locally. Practise on the public seeds instead (`--seeds 1000..1020`). The standings page shows every run, so a bad day on one seed is visible and explained rather than mysterious.